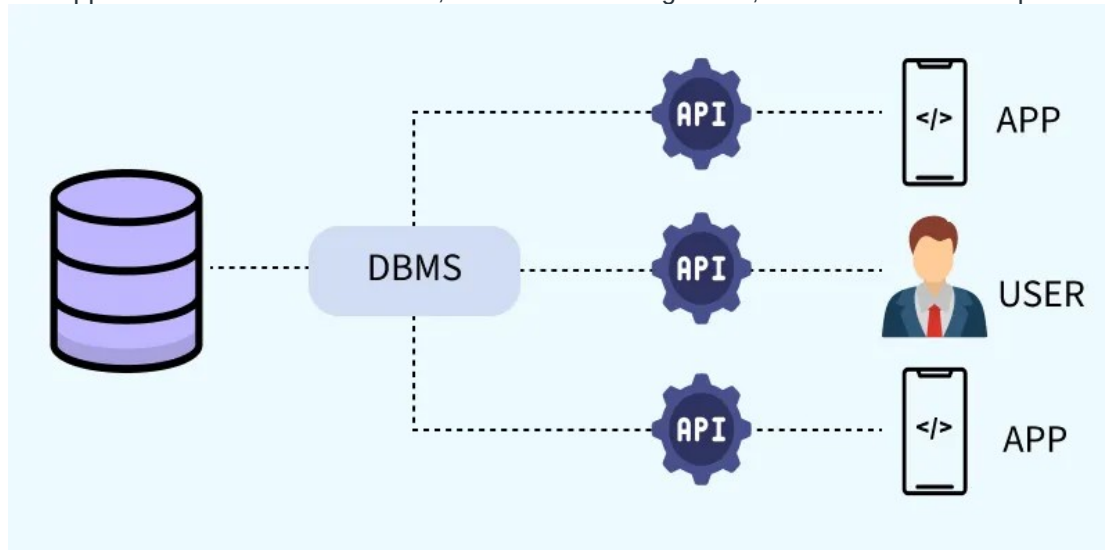


DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems

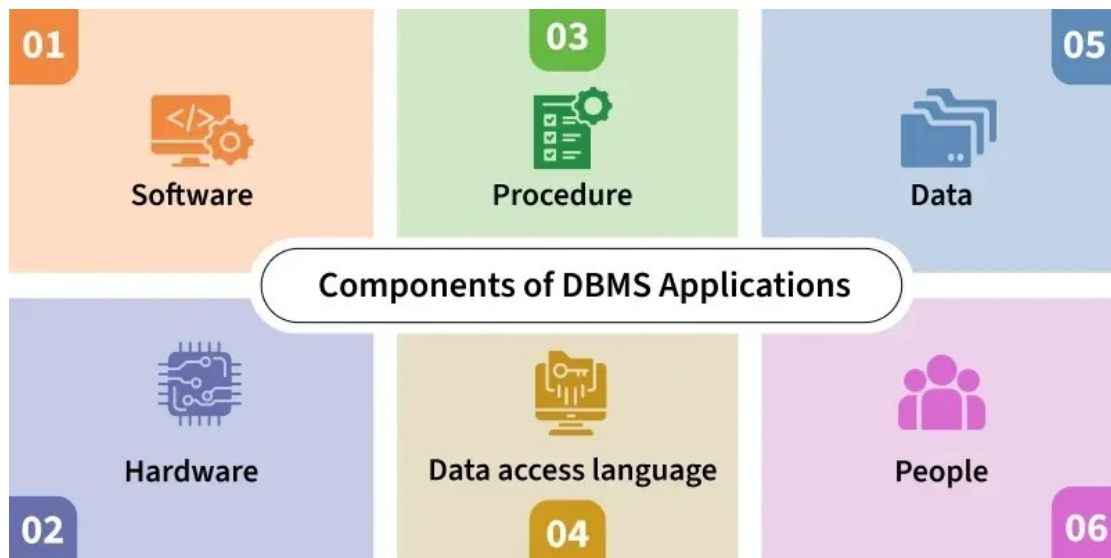
Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Components of DBMS Applications

Any DBMS based applications is made up of six key components that work together to handle data effectively.



Components of DBMS Applications

1. Hardware

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. Software

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. Data

- Raw facts stored in structured or unstructured formats.
- **Operational Data**: Actual user data (e.g., name, age).
- **Metadata**: Data about data (e.g., storage time, size, data type).
- Core reason DBMS exists—to manage and store data efficiently.

4. Procedures

- Instructions and rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

5. Database Access Language

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. People

- Users interacting with DBMS at different levels:
- **Database Administrators (DBA)** – Manage security, performance, user access.
- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Types of DBMS

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. Relational Database Management System (RDBMS)

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL oracle, Microsoft SQL Server and Postgre SQL.

2. NoSQL DBMS

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.
- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Examples: MongoDB, Cassandra, DynamoDB and Redis.

3. Object-Oriented DBMS (OODBMS)

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

Examples: ObjectDB, db4o.

4. Hierarchical Database

- Organizes data in a tree-like structure, where each record (node) has a single parent and have multiple children.
- This model is similar to a file system with folders and subfolders.
- It is efficient for storing data with a clear hierarchy, such as organizational charts or file directories.
- Navigation is fast and predictable due to the fixed structure.
- It lacks flexibility and difficult to restructure or handle complex many-to-many relationships.

Example: IBM Information Management System (IMS).

5. Network Database

- It uses a graph-like model to allow more complex relationships between entities.
- Unlike the hierarchical model, it permits each child to have multiple parents, enabling many-to-many relationships.
- Data is represented using records and sets, where sets define the relationships.
- It is more flexible than the hierarchical model and better suited for applications with complex data linkages.

Example: Integrated Data Store (IDS), TurboIMAGE.

6. Cloud-Based Database

- They are hosted on cloud computing platforms like AWS, Azure or Google Cloud.
- They offer on-demand scalability, high availability, automatic backups and remote accessibility.
- These databases can be relational (SQL) or non-relational (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- They support modern application requirements, including distributed access and real-time analytics.

Example: Amazon RDS (for SQL), MongoDB Atlas (for NoSQL), Google BigQuery.

Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval and security of data. The primary database languages include:

Type of DBMS Language				
Data Definition Language	Data Manipulation Language	Data Control Language	Transaction Control Language	Data Query Language
• CREATE • DROP • TRUNCATE • RENAME • ALTER	• INSERT • DELETE • UPDATE	• REVOKE • GRANT	• ROLLBACK • COMMIT	• SELECT

1. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT:** add comments to the data dictionary
- **RENAME:** rename an object

2. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update and delete data.

- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)
- **CALL:** call a PL/SQL or Java subprogram

- **EXPLAIN PLAN:** interpretation of the data access path
- **LOCK TABLE:** concurrency Control

3. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

4. Transaction Control Language (TCL)

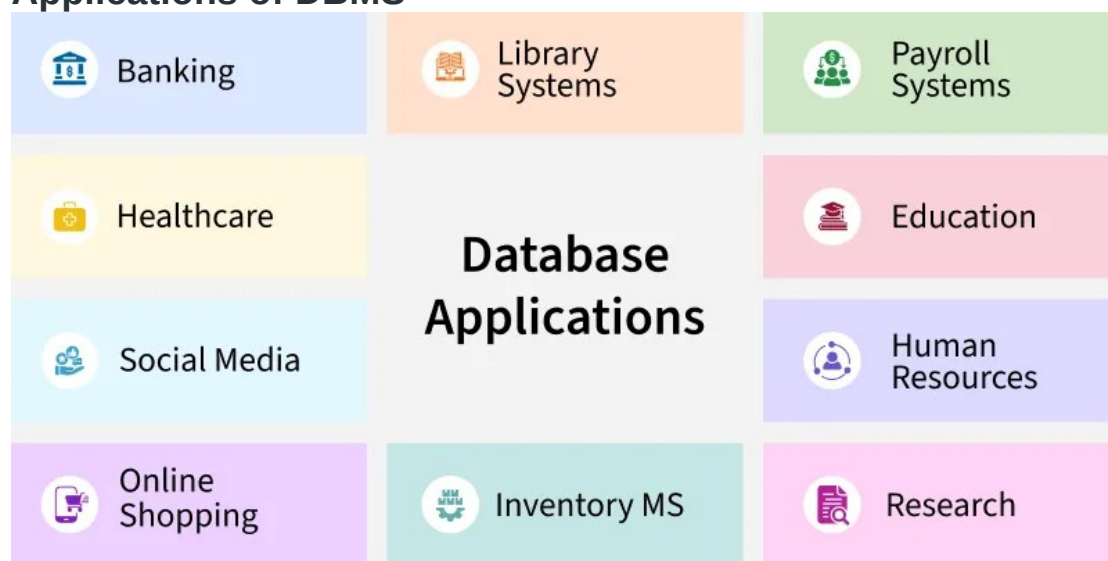
TCL commands oversee transactional data to maintain consistency, reliability and atomicity.

- **ROLLBACK:** Undoes changes made during a transaction.
- **COMMIT:** Saves all changes made during a transaction.
- **SAVEPOINT:** Sets a point within a transaction to which one can later roll back.

5. Data Query Language (DQL)

DQL is a subset of SQL used to retrieve data from a database without modifying it. Its main command is SELECT, which allows users to fetch specific information based on their requirements.

Applications of DBMS



- **Banking:** Manages accounts and transactions.
- **E-commerce:** Tracks products, orders, and customers.
- **Healthcare:** Stores patient records and diagnoses.
- **Education:** Handles student grades and schedules.
- **Social Media:** Manages user profiles and interactions.
- **Data Science:** Supports analytics and predictions.

DBMS Architecture 1-level, 2-Level, 3-Level

A DBMS architecture defines how users interact with the database to read, write, or update information. A well-designed architecture and schema (a blueprint detailing tables, fields and relationships) ensure data consistency, improve performance and keep data secure.

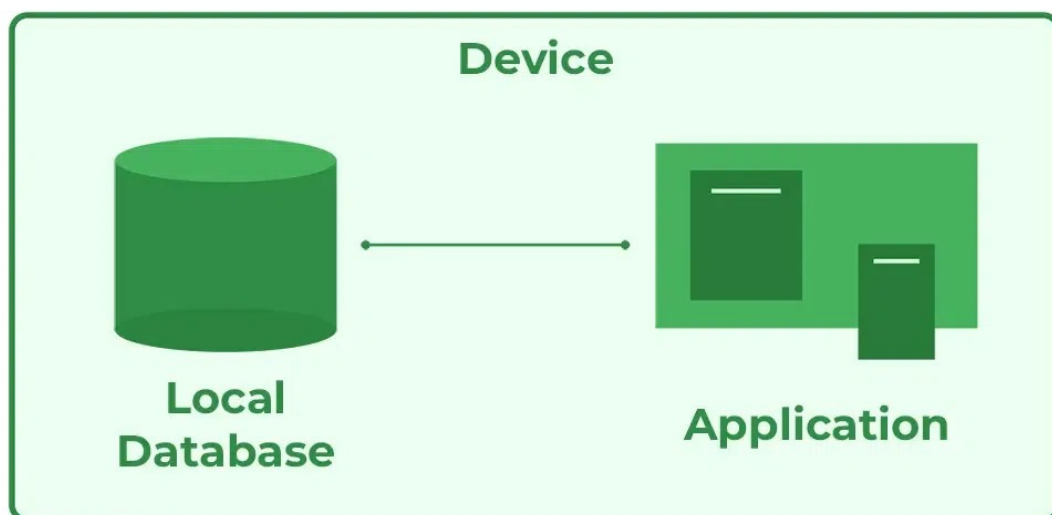
Types of DBMS Architecture

There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the [database](#) on the same system. This means the client, server and database are all in one application. The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.



DBMS 1-Tier Architecture

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- This setup is simple and easy to use, making it ideal for personal or standalone applications. It does not require a network or complex setup, which is why it's often used in small-scale or individual use cases.
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.

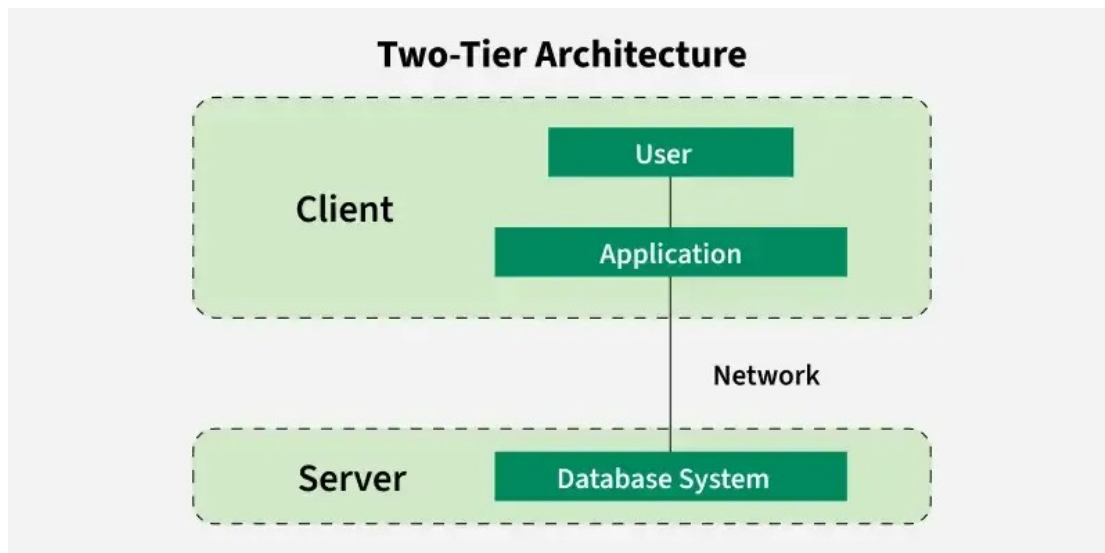
- **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

The 2-tier architecture is similar to a basic [client-server model](#). The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



DBMS 2-Tier Architecture

- On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the server side to communicate with the DBMS. For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.

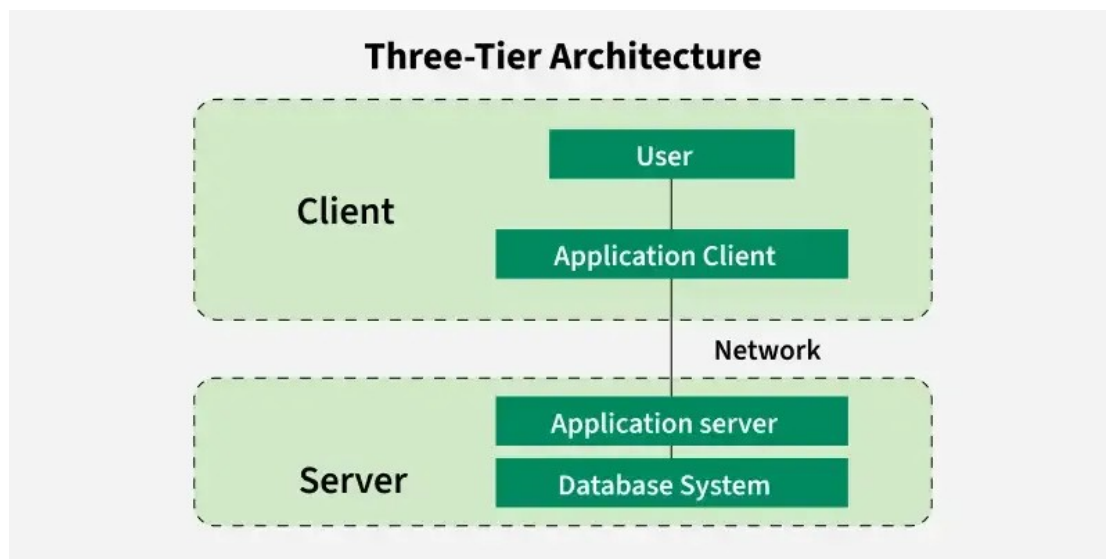
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and [Multi-Tier Architecture](#).
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3-Tier Architecture

In [3-Tier Architecture](#), there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



DBMS 3-Tier Architecture

Example: E-commerce Store

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.
- **Slower Response Time:** Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- **Higher Cost:** Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

Difference between File System and DBMS

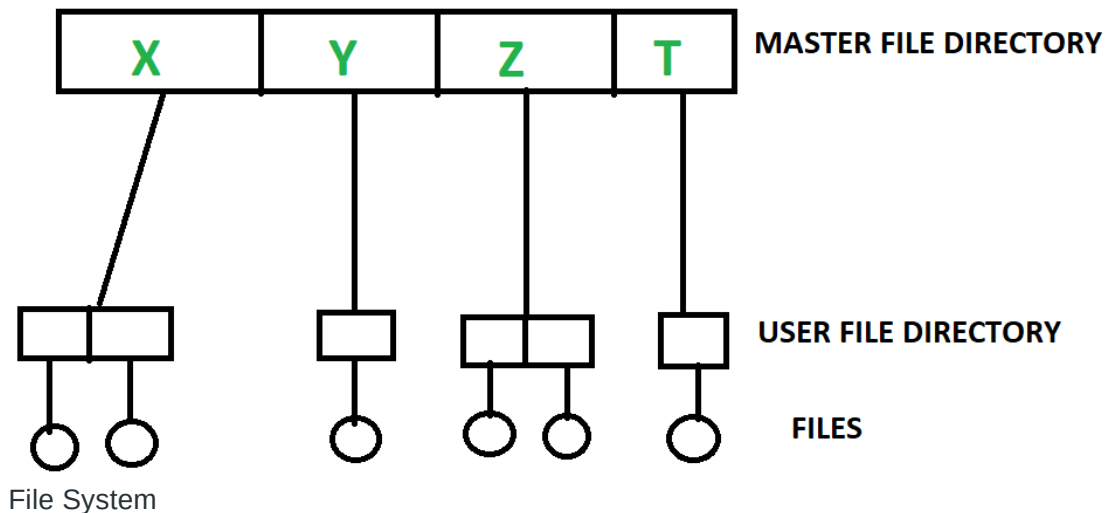
A file system and a DBMS are two kinds of data management systems that are used in different capacities and possess different characteristics. A File System is a way of organizing files into groups and folders and then storing them in a storage device. It provides the media that stores data as well as enables users to perform procedures such as reading, writing, and even erasure.

On the other hand, DBMS is a more elaborate software application that is solely charged with the responsibility of managing large amounts of structured data. It provides functionalities such as query, index, transaction, as well as data integrity. Although the file system serves well for the purpose of data storage for applications where data is to be stored simply and does not require any great organization, DBMS is more appropriate for applications where data needs to be stored and optimized for organizational and structural needs, security, etc.

File System

The **file system** is basically a way of arranging the files in a storage medium like a hard disk. The file system organizes the files and helps in the retrieval of files when they are required. File systems consist of different files which are grouped into directories. The directories further contain other folders and files. The file system performs basic operations like management, file naming, giving access rules, etc.

Example: NTFS(New Technology File System), EXT(Extended File System).

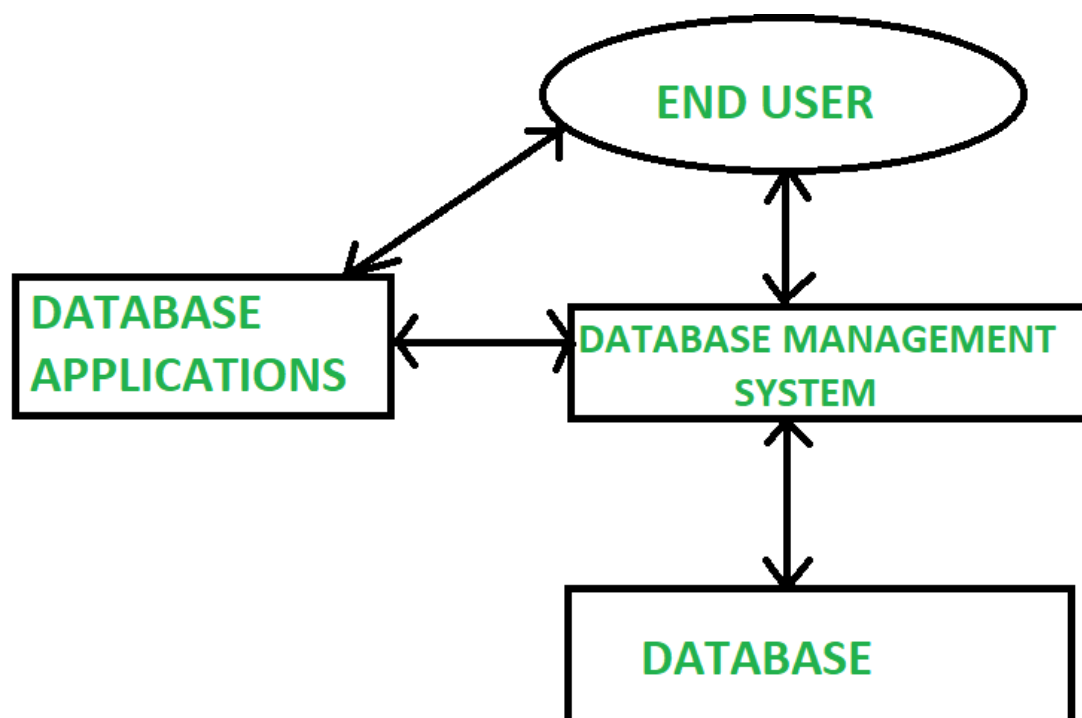


DBMS (Database Management System)

Database Management System is basically software that manages the collection of related data. It is used for storing data and retrieving the data effectively when it is needed. It also provides proper security measures for protecting the data from unauthorized access. In Database Management System the data can be fetched by SQL queries and relational algebra. It also provides mechanisms for data recovery and data backup.

Example:

Oracle, MySQL, MS SQL server.



DBMS

Difference Between File System and DBMS

Basics	File System	DBMS
Structure	The file system is a way of arranging the files in a storage medium within a computer.	DBMS is software for managing the database.
Data Redundancy	Redundant data can be present in a file system.	In DBMS there is no redundant data.
Backup and Recovery	It doesn't provide Inbuilt mechanism for backup and recovery of data if it is lost.	It provides in house tools for backup and recovery of data even if it is lost.
Query processing	There is no efficient query processing in the file system.	Efficient query processing is there in DBMS.
Consistency	There is less data consistency in the file system.	There is more data consistency because of the process of normalization .
Complexity	It is less complex as compared to DBMS.	It has more complexity in handling as compared to the file system.
Security Constraints	File systems provide less security in comparison to DBMS.	DBMS has more security mechanisms as compared to file systems.
Cost	It is less expensive than DBMS.	It has a comparatively higher cost than a file system.
Data Independence	There is no data independence.	In DBMS data independence exists, mainly of two types: 1) Logical Data Independence . 2)Physical Data Independence.
User Access	Only one user can access data at a time.	Multiple users can access data at a time.
Meaning	The users are not required to write procedures.	The user has to write procedures for managing databases
Sharing	Data is distributed in many files. So, it is not easy to share data.	Due to centralized nature data sharing is easy
Data Abstraction	It give details of storage and representation of data	It hides the internal details of Database

Basics	File System	DBMS
Integrity Constraints	Integrity Constraints are difficult to implement	Integrity constraints are easy to implement
Attributes	To access data in a file, user requires attributes such as file name, file location.	No such attributes are required.
Example	Cobol , C++	Oracle , SQL Server

A file system manages storage, while a DBMS provides efficient data management. To learn more, the [GATE CS Self-Paced Course](#) covers these differences thoroughly.

The main difference between a file system and a DBMS (Database Management System) is the way they organize and manage data.

1. File systems are used to manage files and directories, and provide basic operations for creating, deleting, renaming, and accessing files. They typically store data in a hierarchical structure, where files are organized in directories and subdirectories. File systems are simple and efficient, but they lack the ability to manage complex data relationships and ensure data consistency.
2. On the other hand, DBMS is a software system designed to manage large amounts of structured data, and provide advanced operations for storing, retrieving, and manipulating data. DBMS provides a centralized and organized way of storing data, which can be accessed and modified by multiple users or applications. DBMS offers advanced features like data validation, [indexing](#), transactions, [concurrency control](#), and backup and recovery mechanisms. DBMS ensures data consistency, accuracy, and integrity by enforcing data constraints, such as primary keys, foreign keys, and data types.

In summary, file systems are suitable for managing small amounts of [unstructured data](#), while DBMS is designed for managing large amounts of [structured data](#), and offers more advanced features for ensuring data integrity, security, and performance.

Keys in Relational Model

Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.

- They uniquely identify each row in a table.
- They prevent data duplication and maintain consistency.
- They create relationships between different tables.

Different Kinds of Keys	1	Candidate Key	2	Super Key
	3	Alternate Key	4	Foreign Key
	5	Partial Key	6	Primary Key
	7	Secondary Key	8	Unique Key
	9	Composite Key	10	Surrogate Key

Keys in DBMS

Importance of Keys in DBMS

Keys are important in a Database Management System (DBMS) for several reasons:

- **Uniqueness:** Keys ensure that each record in a table is unique and can be identified distinctly.
- **Data Integrity:** Keys prevent data duplication and maintain the consistency of the data.
- **Efficient Data Retrieval:** Keys help in creating relationships between tables, allowing faster queries and better data organization.

Without keys, managing large databases would become difficult, and data retrieval would be slow and error-prone.

Types of Database Keys

1. Super Key

The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as [Super Key](#). It may include extra attributes that aren't important for uniqueness but still uniquely identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.

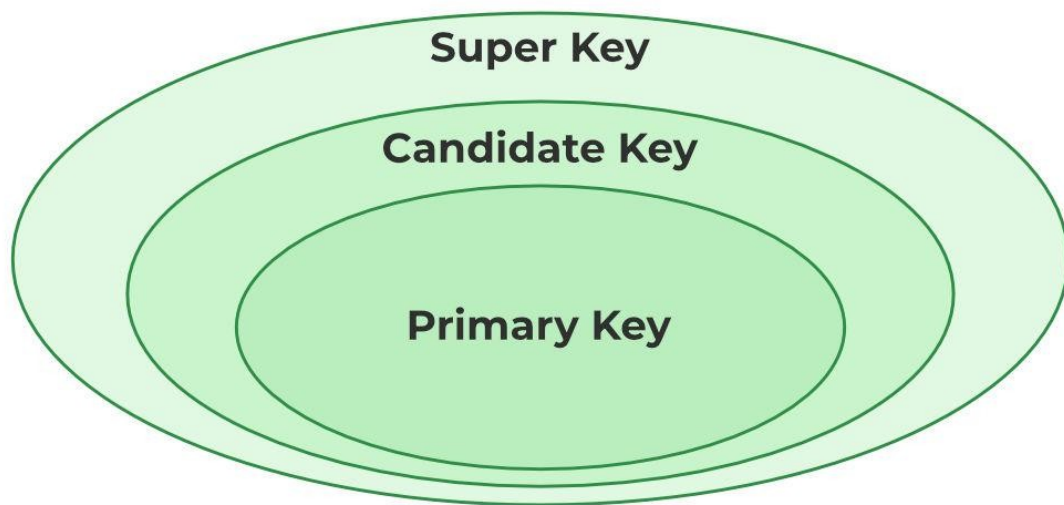
- A super key is a group of single or multiple keys that uniquely identifies rows in a table. It supports NULL values in rows.
- A super key can contain extra attributes that aren't necessary for uniqueness.
- For example, if the "STUD_NO" column can uniquely identify a student, adding "SNAME" to it will still form a valid super key, though it's unnecessary.

Example: Consider the STUDENT table

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

A super key could be a combination of STUD_NO and PHONE, as this combination uniquely identifies a student.



Relation between Primary Key, Candidate Key, and Super Key

2. Candidate Key

The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- A candidate key is a minimal super key, meaning it can uniquely identify a record but contains no extra attributes.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.
- A candidate key must contain unique values, ensuring that no two rows have the same value in the candidate key's columns.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only one primary key.

Example: For the STUDENT table below, STUD_NO can be a candidate key, as it uniquely identifies each record.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

Table: STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

A composite candidate key example: {STUD_NO, COURSE_NO} can be a candidate key for a STUDENT_COURSE table.

3. Primary Key

A [primary key](#) is chosen from the set of candidate keys to uniquely identify each record in a table. For example, in the STUDENT table, both STUD_NO and STUD_PHONE can be candidate keys, but STUD_NO is selected as the primary key.

- It uniquely identifies every tuple (row) and does not allow duplicate values.
- It cannot be NULL, as each record must have a valid identifier.
- It may be single-column or composite (made of multiple columns).
- Databases often organize data using the primary key to allow faster access and searching.

Example: The STUDENT table has the structure Student(STUD_NO, SNAME, ADDRESS, PHONE), where STUD_NO is the primary key.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

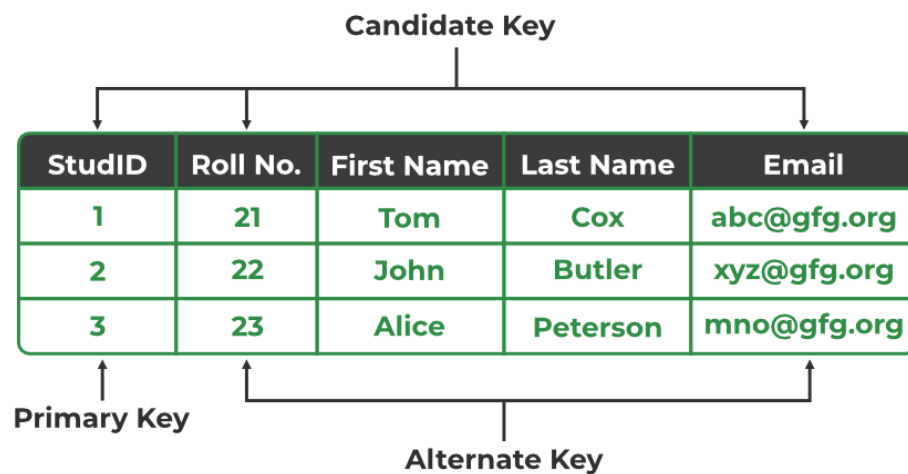
STUDENT Table

4. Alternate Key

An [alternate key](#) is any candidate key in a table that is not chosen as the primary key. In other words, all the keys that are not selected as the primary key are considered alternate keys.

- An alternate key is also referred to as a secondary key because it can uniquely identify records in a table, just like the primary key.
- An alternate key can consist of one or more columns (fields) that can uniquely identify a record, but it is not the primary key

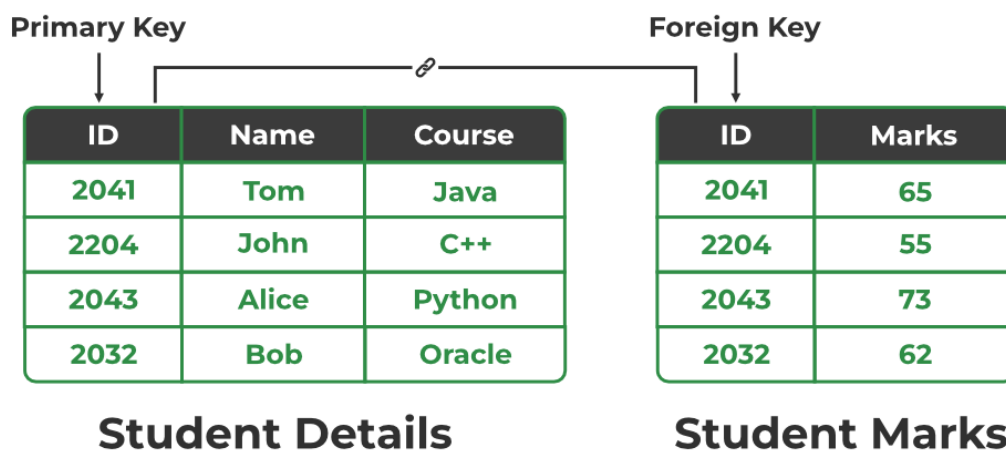
Example: In the STUDENT table, both STUD_NO and PHONE are candidate keys. If STUD_NO is chosen as the primary key, then PHONE would be considered an alternate key.



Primary Key, Candidate Key, and Alternate Key

5. Foreign Key

A [foreign key](#) is an attribute in one table that refers to the primary key in another table. The table that contains the foreign key is called the referencing table and the table that is referenced is called the referenced table.



Relation between Primary Key and Foreign Key

- A foreign key in one table points to the primary key in another table, establishing a relationship between them.
- It helps connect two or more tables, enabling you to create relationships between them. This is important for maintaining data integrity and preventing data redundancy.
- They act as a cross-reference between the tables.

Example: Consider the STUDENT_COURSE table

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

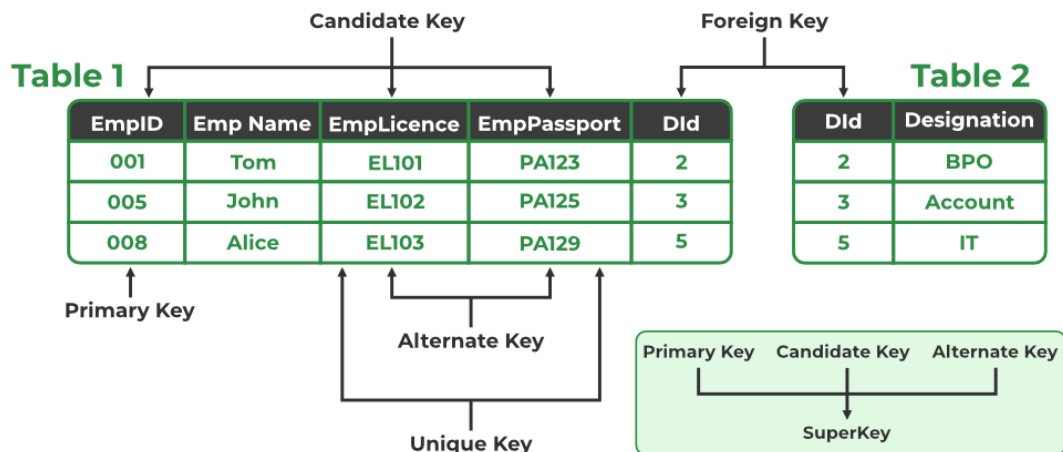
- STUD_NO in the STUDENT_COURSE table is a foreign key that refers to the STUD_NO primary key of the STUDENT table.
- Unlike a primary key, a foreign key can contain duplicate values and may be NULL. For example, STUD_NO appears multiple times in STUDENT_COURSE because a student can enroll in more than one course.
- However, STUD_NO in the STUDENT table is a primary key, so it must always be unique and non-NULL.

6. Composite Key

Sometimes, a single column is not enough to uniquely identify all records in a table, so a combination of multiple attributes is used. An optimal set of such attributes is chosen to ensure that every row is uniquely identifiable.

- It acts as a primary key if there is no primary key in a table
- Two or more attributes are used together to make a composite key.
- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

Example: In the STUDENT_COURSE table, {STUD_NO, COURSE_NO} can form a composite key to uniquely identify each record.



Different Types of Keys

Introduction of Relational Algebra in DBMS

Relational Algebra is a formal language used to query and manipulate relational databases, consisting of a set of operations like selection, projection, union, and join. It provides a mathematical framework for querying databases, ensuring efficient data retrieval and manipulation.

- Relational algebra serves as the mathematical foundation for query SQL.
- SQL queries are based on relational algebra operations, enabling users to retrieve data effectively.

Note: Relational algebra simplifies the process of querying databases and makes it easier to understand and optimize query execution for better performance. It is essential for learning SQL

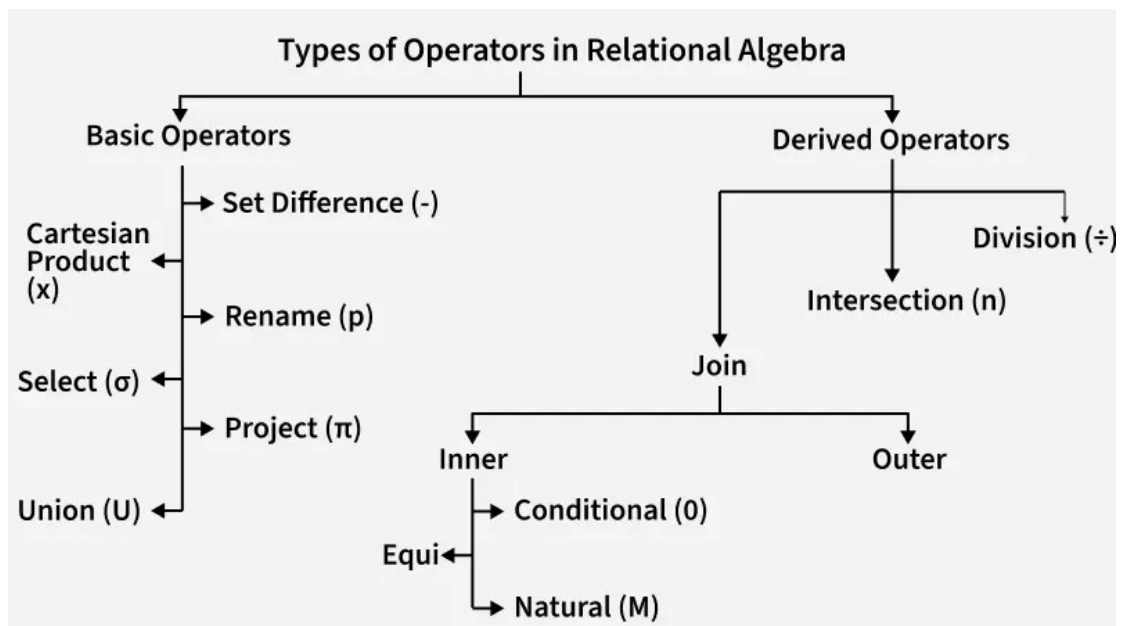
Key Concepts in Relational Algebra

Before explaining relational algebra operations, let's define some fundamental concepts:

- **Relations:** In relational algebra, a relation is a table that consists of rows and columns, representing data in a structured format. Each relation has a unique name and is made up of tuples.
- **Tuples:** A tuple is a single row in a relation, which contains a set of values for each attribute. It represents a single data entry or record in a relational table.
- **Attributes:** Attributes are the columns in a relation, each representing a specific characteristic or property of the data. For example, in a "Students" relation, attributes could be "Name", "Age", and "Grade".
- **Domains:** A domain is the set of possible values that an attribute can have. It defines the type of data that can be stored in each column of a relation, such as integers, strings, or dates.

Basic Operators in Relational Algebra

Relational algebra consists of various basic operators that help us to fetch and manipulate data from relational tables in the database to perform certain operations on relational data. Basic operators are fundamental operations that include [selection \(\$\sigma\$ \)](#), [projection \(\$\pi\$ \)](#), [union \(\$\cup\$ \)](#), [set difference \(\$-\$ \)](#), [Cartesian product \(\$\times\$ \)](#), and [rename \(\$\rho\$ \)](#).



Operators in Relational Algebra

1. Selection(σ)

The [Selection Operation](#) is basically used to filter out rows from a given table based on certain given condition. It basically allows us to retrieve only those rows that match the condition as per condition passed during [SQL Query](#).

Example: If we have a relation R with attributes A, B, and C, and we want to select tuples where $C > 3$, we write:

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

$\sigma_{(c>3)}(R)$ will select the tuples which have c more than 3.

Output:

A	B	C
1	2	4
4	3	4

Explanation: The selection operation only filters rows but does not display or change their order. The projection operator is used for displaying specific columns.

2. Projection(π)

While Selection operation works on rows, similarly projection operation of relational algebra works on columns. It basically allows us to pick specific columns from a given relational table based on the given condition and ignoring all the other remaining columns.

Example: Suppose we want columns B and C from Relation R.

$\pi_{(B,C)}(R)$ will show following columns.

Output:

B	C
2	4
2	3
3	4

Explanation: By Default, projection operation removes duplicate values.

3. Union(U)

The Union Operator is basically used to combine the results of two queries into a single result. The only condition is that both queries must return same number of columns with same [data types](#). Union operation in relational algebra is the same as union operation in [set theory](#).

Example: Consider the following table of Students having different optional subjects in their course.

FRENCH

Student_Name	Roll_Number
Ram	01
Mohan	02
Vivek	13
Geeta	17

GERMAN

Student_Name	Roll_Number
Vivek	13
Geeta	17
Shyam	21
Rohan	25

If FRENCH and GERMAN relations represent student names in two subjects, we can combine their student names as follows:

$$\pi(\text{Student_Name})(\text{FRENCH}) \cup \pi(\text{Student_Name})(\text{GERMAN})$$

Output:

Student_Name
Ram
Mohan
Vivek
Geeta
Shyam
Rohan

Explanation: The only constraint in the union of two relations is that both relations must have the same set of Attributes.

4. Set Difference(-)

Set difference basically provides the rows that are present in one table, but not in another tables. Set Difference in relational algebra is the same [set difference operation](#) as in set theory.

Example: To find students enrolled only in FRENCH but not in GERMAN, we write:

$$\pi(\text{Student_Name})(\text{FRENCH}) - \pi(\text{Student_Name})(\text{GERMAN})$$

Student_Name
Ram
Mohan

Explanation: The only constraint in the Set Difference between two relations is that both relations must have the same set of Attributes.

5. Rename(ρ)

Rename operator basically allows you to give a temporary name to a specific relational table or to its columns. It is very useful when we want to avoid ambiguity, especially in complex Queries. Rename is a unary operation used for renaming attributes of a relation.

Example: We can rename an attribute B in relation R to D

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

$\rho_{(D/B)}R$ will rename the attribute 'B' of the relation by "D".

Output Table:

A	D	C
1	2	4
2	2	3
3	2	3
4	3	4

6. Cartesian Product(X)

The Cartesian product combines every row of one table with every row of another table, producing all the possible combination. It's mostly used as a precursor to more complex operation like joins. Let's say A and B, so the cross product between A X B will result in all the attributes of A followed by each attribute of B. Each record of A will pair with every record of B.

Relation A:

Name	Age	Sex
Ram	14	M
Sona	15	F
Kim	20	M

Relation B:

ID	Course
1	DS
2	DBMS

Output: If relation A has 3 rows and relation B has 2 rows, the Cartesian product A × B will result in 6 rows.

Name	Age	Sex	ID	Course
Ram	14	M	1	DS
Ram	14	M	2	DBMS
Sona	15	F	1	DS
Sona	15	F	2	DBMS
Kim	20	M	1	DS
Kim	20	M	2	DBMS

Explanation: If A has 'n' tuples and B has 'm' tuples then A X B will have 'n*m' tuples.

Basic Operators in Relational Algebra

Table 1: STUDENT_SPORTS

ROLL_N O	SPORTS
1	Badminton
2	Cricket

2	Badminton
4	Badminton

Table 2: EMPLOYEE

EMP_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
5	NARESH	HISAR	9782918192	22
6	SWETA	RANCHI	9852617621	21
4	SURESH	DELHI	9156768971	18

Table 3: STUDENT

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI	9156768971	18

1. Selection operator (σ)

Selection operator is used to selecting tuples from a relation based on some condition.

Syntax:

$\sigma_{(Condition)}(Relation\ Name)$

Extract students whose age is greater than 18 from STUDENT relation given in Table 3

$\sigma_{(AGE>18)}(STUDENT)$

Note: SELECT operation does not show any result, the projection operator must be called before the selection operator to generate or project the result. So, the correct syntax to generate the result is:

$\Pi(\sigma_{(AGE>18)}(STUDENT))$

Result:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
---------	------	---------	-------	-----

3	SUJIT	ROHTAK	9156253131	20
---	-------	--------	------------	----

2. Projection Operator (Π)

Projection operator is used to project particular columns from a relation.

Syntax:

$\Pi(\text{Column 1, Column 2....Column n})(\text{Relation Name})$

Extract ROLL_NO and NAME from STUDENT relation given in Table 3

$\Pi(\text{ROLL_NO, NAME})(\text{STUDENT})$

Result:

ROLL_NO	NAME
1	RAM
2	RAMESH
3	SUJIT
4	SURESH

Note: If the resultant relation after projection has duplicate rows, it will be removed. For Example $\Pi(\text{ADDRESS})(\text{STUDENT})$ will remove one duplicate row with the value DELHI and return three rows.

3. Cross Product(X)

Cross product is used to join two relations. For every row of Relation1, each row of Relation2 is concatenated. If Relation1 has m tuples and Relation2 has n tuples, cross product of Relation1 and Relation2 will have m x n tuples.

Syntax:

Relation1 X Relation2

To apply Cross Product on STUDENT relation given in Table 1 and STUDENT_SPORTS relation given in Table 2,

STUDENT X STUDENT_SPORTS

Result:

ROLL_NO	NAME	ADDRESS	PHONE	AGE	ROLL_NO	SPORTS
1	RAM	DELHI	9455123451	18	1	Badminton
1	RAM	DELHI	9455123451	18	2	Cricket

ROLL_N O	NAME	ADDRE SS	PHONE	AG E	ROLL_N O	SPORTS
1	RAM	DELHI	9455123451	18	2	Badminton
1	RAM	DELHI	9455123451	18	4	Badminton
2	RAMES H	GURGAO N	9652431543	18	1	Badminton
2	RAMES H	GURGAO N	9652431543	18	2	Cricket
2	RAMES H	GURGAO N	9652431543	18	2	Badminton
2	RAMES H	GURGAO N	9652431543	18	4	Badminton
3	SUJIT	ROHTAK	9156253131	20	1	Badminton
3	SUJIT	ROHTAK	9156253131	20	2	Cricket
3	SUJIT	ROHTAK	9156253131	20	2	Badminton
3	SUJIT	ROHTAK	9156253131	20	4	Badminton
4	SURES H	DELHI	9156768971	18	1	Badminton
4	SURES H	DELHI	9156768971	18	2	Cricket
4	SURES H	DELHI	9156768971	18	2	Badminton
4	SURES H	DELHI	9156768971	18	4	Badminton

4. Union (U)

Union on two relations R1 and R2 can only be computed if R1 and R2 are **union compatible** (These two relations should have the same number of attributes and corresponding attributes in two relations have the same domain). Union operator when applied on two relations R1 and R2 will give a relation with tuples that are either in R1 or in R2. The tuples which are in both R1 and R2 will appear only once in the result relation.

Syntax:

Relation1 U Relation2

Find the person who is either student or employees, we can use Union operators like:

STUDENT U EMPLOYEE

RESULT:

ROLL_N	NAME	ADDRES	PHONE	AG
--------	------	--------	-------	----

O		S		E
1	RAM	DELHI	9455123451	18
2	RAMES H	GURGA ON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURES H	DELHI	9156768971	18
5	NARES H	HISAR	9782918192	22
6	SWETA	RANCHI	9852617621	21

5. Minus (-) or Set Difference

Minus on two relations R1 and R2 can only be computed if R1 and R2 are **union compatible**. Minus operator when applied on two relations as R1-R2 will give a relation with tuples that are in R1 but not in R2. Syntax:

Relation1 - Relation2

Find the person who is a student but not an employee, we can use minus operator like:

STUDENT - EMPLOYEE

Result:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
2	RAMES H	GURGA ON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20

6. Rename(ρ)

Rename operator is used to rename a relation or its attributes for clarity or further operations.

Syntax:

ρ (new_relation_name, original_relation)

To rename STUDENT relation to STUDENT1, we can use rename operator like:

ρ (STUDENT1, STUDENT)

If you want to create a relation STUDENT_NAMES with ROLL_NO and NAME from STUDENT, it can be done using rename operator as:

ρ (STUDENT_NAMES, Π (ROLL_NO, NAME)(STUDENT))

Extended Relational Algebra Operators

- Intersection ($\cap$)
- Division ($\div$)
- Join Operations ($\bowtie$)
- [Natural Join](#)
- Theta Join
- Equi Join

These operators provide more functionality for complex queries in relational databases.

Derived Operators in Relational Algebra

Derived operators are built using basic operators and include operations like join, intersection, and division. These operators help perform more complex queries by combining basic operations to meet specific data retrieval needs.

1. Join Operators

Join operations in relational algebra combine data from two or more relations based on a related attribute, allowing for more complex queries and data retrieval. Different types of joins include:

1.1 Inner Join

An inner join combines rows from two relations based on a matching condition and only returns rows where there is a match in both relations. If a record in one relation doesn't have a corresponding match in the other, it is excluded from the result. This is the most common type of join.

a. Conditional Join:

- A conditional join is an inner join where the matching condition can involve any comparison operator like equals ($=$), greater than ($>$), etc.
- **Example:** Joining Employees and Departments on DepartmentID where Salary $>$ 50000 will return employees in departments with a salary greater than 50,000

b. Equi Join:

- An equi join is a type of conditional join where the condition is specifically equality ($=$) between columns from both relations.
- **Example:** Joining Customers and Orders on CustomerID where both relations have this column, returning only matching records.

c. Natural Join:

- A natural join automatically combines relations based on columns with the same name and type, removing duplicate columns in the result. It's a more efficient way of joining.
- **Example:** Joining Students and Enrollments where StudentID is common in both, and the result contains only unique columns.

1.2 Outer Join

An outer join returns all rows from one relation, and the matching rows from the other relation. If there is no match, the result will still include all rows from the outer relation with NULL values in the columns from the unmatched relation.

a. Left Outer Join:

- A left outer join returns all rows from the left relation and the matching rows from the right relation.
- If there is no match, the result will include NULL values for the right relation's attributes.
- **Example:** Joining Employees with Departments using a left outer join ensures all employees are listed, even those who aren't assigned to any department, with NULL values for the department columns.

b. Right Outer Join:

- A right outer join returns all rows from the right relation and the matching rows from the left relation.
- If no match exists, the left relation's columns will contain NULL values.
- **Example:** Joining Departments with Employees using a right outer join includes all departments, even those with no employees assigned, filling unmatched employee columns with NULL.

c. Full Outer Join:

- A full outer join returns all rows when there is a match in either the left or right relation.
- If a row from one relation does not have a match in the other, NULL values are included for the missing side.
- **Example:** Joining Customers and Orders using a full outer join will return all customers and orders, even if there's no corresponding order for a customer or no customer for an order.

2. Set Intersection($\cap$)

[Set Intersection](#) basically allows to fetches only those rows of data that are common between two sets of relational tables. Set Intersection in relational algebra is the same set intersection operation in set theory.

Example: Consider the following table of Students having different optional subjects in their course.

Relation FRENCH

Student_Name	Roll_Number
Ram	01
Mohan	02
Vivek	13
Geeta	17

Relation GERMAN

Student_Name	Roll_Number
Vivek	13
Geeta	17
Shyam	21

Student_Name	Roll_Number
Rohan	25

From the above table of FRENCH and GERMAN, the Set Intersection is used as follows:

$$\pi(\text{Student_Name})(\text{FRENCH} \cap \pi(\text{Student_Name})(\text{GERMAN}))$$

Output:

Student_Name
Vivek
Geeta

Explanation: The only constraint in the Set Difference between two relations is that both relations must have the same set of Attributes.

3. Division (÷)

The Division Operator is used to find tuples in one relation that are related to all tuples in another relation. It's typically used for "for all" queries.

Student_Course (Dividend Table):

Student_ID	Course_ID
101	C1
101	C2
102	C1
103	C1
103	C2

Course (Divisor Table):

Course_ID
C1
C2

Example: Query is to find students who are enrolled in all courses listed in the Course table. In this case, students must be enrolled in both C1 and C2.

$$\text{Student_Course}(\text{Student_ID}, \text{Course_ID}) \div \text{Course}(\text{Course_ID})$$

Output:

Student_ID
101
103

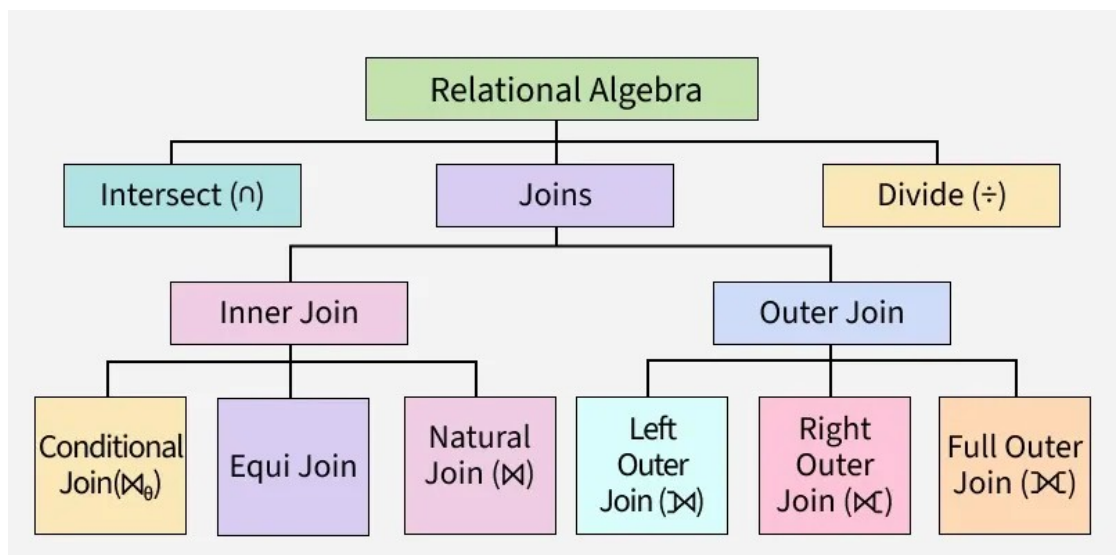
Extended operators in relational algebra are operators that go beyond the basic set of relational algebra operations. They are also known as derived operators because they can be constructed from combinations of the fundamental operators.

There are mainly three types of extended operators in Relational Algebra:

Join

Intersection ($\cap$)

Divide ($\div$)



We will be explaining these types using the following tables:

Table R:

A	B
1	x
2	y
3	z

Table S:

B	C
x	10
y	20
w	30

Join

Join operators in DBMS are used to combine data from two or more tables based on a related column between them. These operators allow efficient data retrieval, making it easier to perform complex queries. The most common types of Join operators are Inner Join and Outer Join.

1. Inner Join

Inner join returns rows when there is a match in both tables. If there is no match, the row is excluded from the result. It is the most frequently used join in relational databases and ensures that only matching records from both tables are included.

Inner joins can be categorized into more specific types based on how the join condition is defined:

1.1 Conditional Join($\bowtie_{\theta}$): Conditional Join or Theta Join is used when you want to join two or more relation based on some conditions. It supports various comparison operators, such as $<$, $>$, $<=$, $>=$, $=$ and $\neq$.

Example: Join tables R and S based on a condition θ . For example, join where $R.B = S.B$ and $R.A > 1$.

$R \bowtie_{R.B=S.B \wedge R.A>1} S$

Output Table:

A	R.B	S.B	C
2	y	y	20

Note: The selection operator only selects the required tuples but does not display them. For display, the data projection operator is used.

1.2 Equi Join: Equi Join is a special case of conditional join where only equality condition holds between a pair of attributes. As values of two attributes will be equal in result of equijoin, only one attribute will be appeared in result.

Example: Join tables R and S where $R.B = S.B$.

$R \bowtie_{R.B=S.B} S$

Output Table:

A	R.B	S.B	C
1	x	x	10
2	y	y	20

1.3 Natural Join($\bowtie$): A Natural Join automatically combines two tables based on matching column names and data type, eliminating duplicate columns and providing a seamless result set. While applying natural join on two relations, there is no need to write equality condition explicitly. Natural Join will also return the similar attributes only once as their value will be same in resulting relation.

Example: Join tables R and S on the common attribute B and eliminate duplicate columns.

$R \bowtie S$

Output Table:

A	B	C
1	x	10
2	y	20

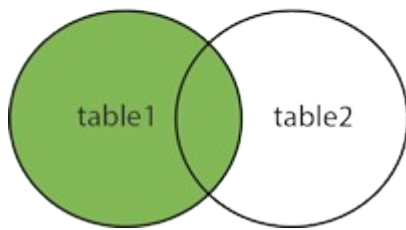
Natural Join is by default inner join because the tuples which does not satisfy the conditions of join does not appear in result set.

2. Outer Join

The Outer Join returns all records from one table and the matched records from the other table. If no match is found, the result will include NULL values for the non-matching columns. Outer joins can be further classified into Left Outer Join, Right Outer Join and Full Outer Join, based on which table's records are prioritized in case of non-matching rows.

2.1 Left Outer Join($\bowtie\rightarrow$): A Left Outer Join in DBMS returns all records from the left table and the matching records from the right table. If there is no match in the right table, it still includes all rows from the left table with NULL values for the columns of the right table.

LEFT JOIN



Left Outer Join

Example: Join tables R and S on R.B = S.B and include all rows from R even if there is no match in S.

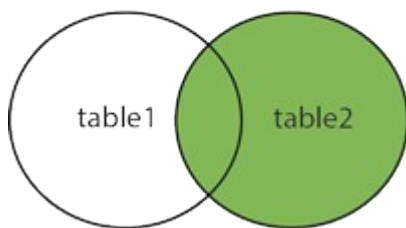
$R \bowtie S$

Output Table:

A	R.B	S.B	C
1	x	x	10
2	y	y	20
3	z	NUL L	NUL L

2.2 Right Outer Join($\bowtie$): A Right Outer Join retrieves all records from the right table and the matching records from the left table. If there is no match in the left table, the result will still include all rows from the right table, with NULL values for the left table's columns. This join is particularly useful when you want to ensure all data from the right table is included, even if no corresponding records exist in the left table.

RIGHT JOIN



Right Outer Join

Example: Join tables R and S on R.B = S.B and include all rows from S even if there is no match in R.

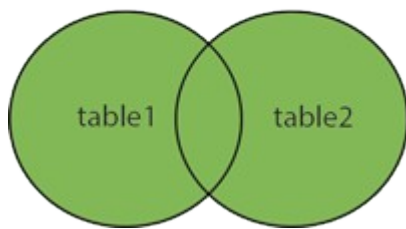
$R \bowtie S$

Output Table:

A	R.B	S.B	C
1	x	x	10
2	y	y	20
NUL L	NUL L	w	30

2.3 Full Outer Join($\bowtie$): A Full Outer Join returns all records when there is a match in either the left or right table. If there is no match, it includes all rows from both tables with NULL values for the missing side. This join is useful when you need to ensure that no data is lost from either table, making it ideal for combining datasets where all information should be preserved, regardless of whether a match exists.

FULL OUTER JOIN



Full Outer Join

Example: Join tables R and S on R.B = S.B and include all rows from both tables, filling in NULLs where there is no match.

$R \bowtie S$

Output Table:

A	R.B	S.B	C
1	x	x	10
2	y	y	20
3	z	NUL L	NUL L
NUL L	NUL L	w	30

Intersection ($\cap$)

Intersection is an operator that returns the common records from two relations. It retrieves rows that appear in both tables, ensuring that only the matching data from both sets is included in the result. Intersection on two relations can only be computed if both relations are union compatible.

It means two relation should have same number of attributes and corresponding attributes in two relations have same domain. In simple words, both table should have same schema.

Example: Assume R and S are as follows for intersection:

Table R:

A	B
1	x
2	y
3	z

Table S:

A	B
1	x
2	y

Both R and S have the same schema (A and B). The intersection of R and S returns rows that are present in both R and S.

$R \cap S$

Output Table:

A	B
1	x
2	y

Division ($\div$)

The Division operator is used to find records in one relation that are associated with all records in another relation. It is commonly used when we want to identify entities that satisfy certain conditions across multiple related data sets.

The Division operator ($R \div S$) can be applied if:

The attributes of B are a proper subset of the attributes of R.

The result will include all attributes of A except those that are in S.

It returns the tuples from R that are associated with every tuple in S.

Example: Assume R and S are as follows for division,

Table R:

A	B
1	x
1	y
2	x
2	y
3	z

Table S:

B
x
y

The division $R \div S$ returns values of A that are associated with all values of B in S.

$R \div S$

Output Table:

A
1
2

Relational Calculus

Relational calculus is a non-procedural query language used in the context of relational algebra. It focuses on what data to retrieve, rather than how to retrieve it, making it different from relational algebra, which is procedural. In relational calculus, queries are expressed using logical formulas that describe the desired result, without specifying the exact steps to get there.

SQL Joins Implementation (Inner, Left, Right and Full Join)

SQL Joins are used to combine rows from two or more tables based on a related column between them. They help in retrieving connected data stored across multiple tables.

Types of SQL Joins

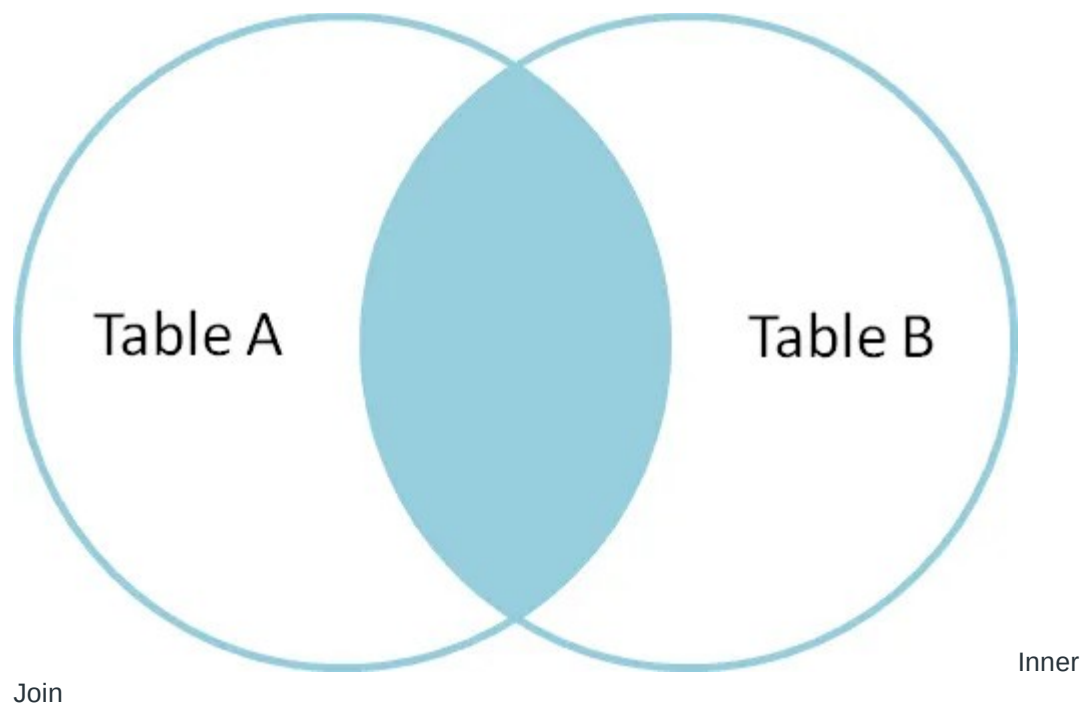
SQL joins are categorized into different types based on how rows from two tables are matched and combined.

1. SQL INNER JOIN

The [INNER JOIN](#) in SQL returns only the rows where there is a matching value in both tables. It combines records based on a related column and excludes non-matching rows.

Syntax:

```
SELECT  
table1.column1,table1.column2,table2.column1,....  
FROM table1 INNER JOIN  
table2 ON table1.matching_column =  
table2.matching_column;
```

Note: We can also write *JOIN* instead of *INNER JOIN*. *JOIN* is same as *INNER JOIN*.

Example of INNER JOIN:

Consider the two tables, Student and StudentCourse, which share a common column `ROLL_NO`. Using SQL JOINS, we can combine data from these tables based on their relationship, allowing us to retrieve meaningful information like student details along with their enrolled courses.

S_ID	NAME	ADDRESS
1	Liam	London
2	Sofia	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

Students Table

COURSE_ID	ROLL_NO
1	1
2	2
3	3
6	7

StudentCourse Table

Let's look at the example of INNER JOIN clause, and understand it's working. This query will show the names and age of students enrolled in different courses.

Query:

```
SELECT StudentCourse.COURSE_ID, Student.NAME,
Student.AGE FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Output:

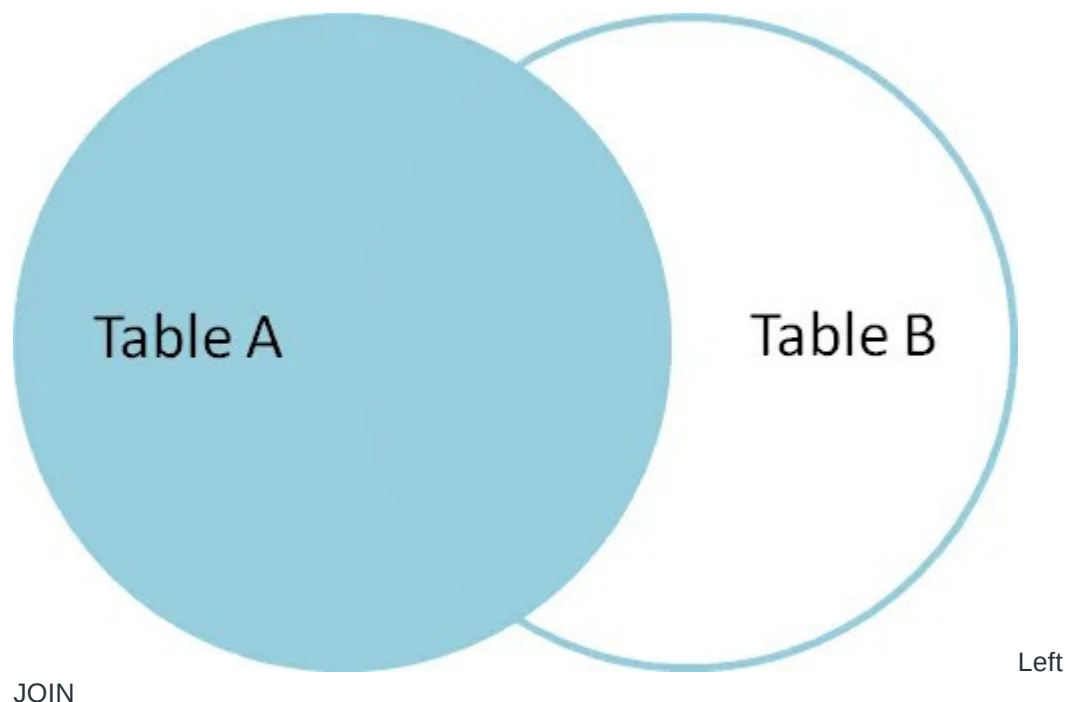
COURSE_ID	NAME	AGE
1	Liam Johnson	18
2	Emma Wilson	19
3	Noah Smith	20

2. SQL LEFT JOIN

A LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table. [LEFT JOIN](#) is also known as LEFT OUTER JOIN.

Syntax:

```
SELECT
table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Note: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.

Example: In this example, the LEFT JOIN retrieves all rows from the Student table and the matching rows from the StudentCourse table based on the ROLL_NO column.

Query:

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

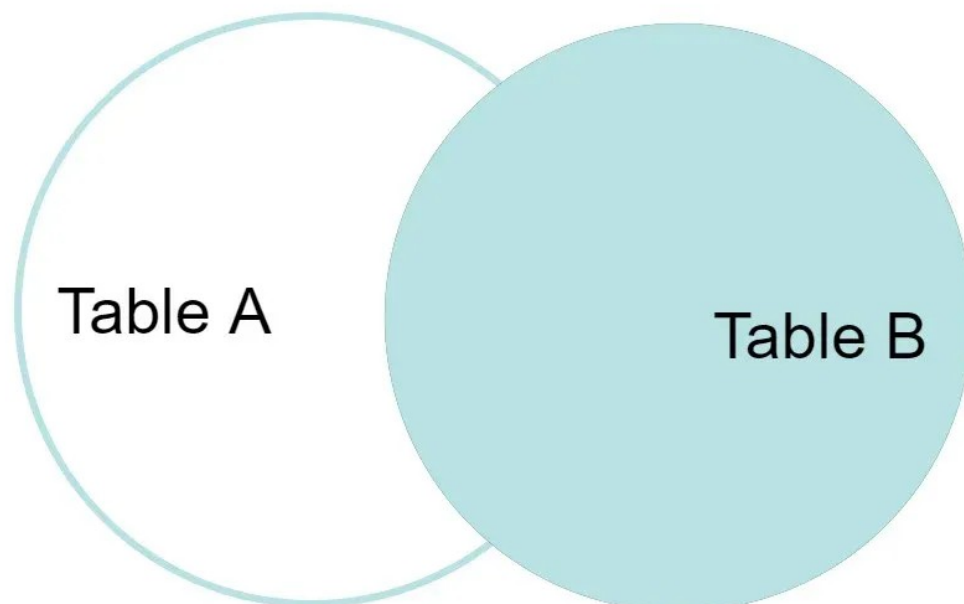
NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	

3. SQL RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null. RIGHT JOIN is also known as RIGHT OUTER JOIN.

Syntax

```
SELECT
table1.column1, table1.column2, table2.column1, ....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Right JOIN

Note: We can also use RIGHT OUTER JOIN instead of RIGHT JOIN, both are the same

Example: In this example, the RIGHT JOIN retrieves all rows from the StudentCourse table and the matching rows from the Student table based on the ROLL_NO column.

Query:

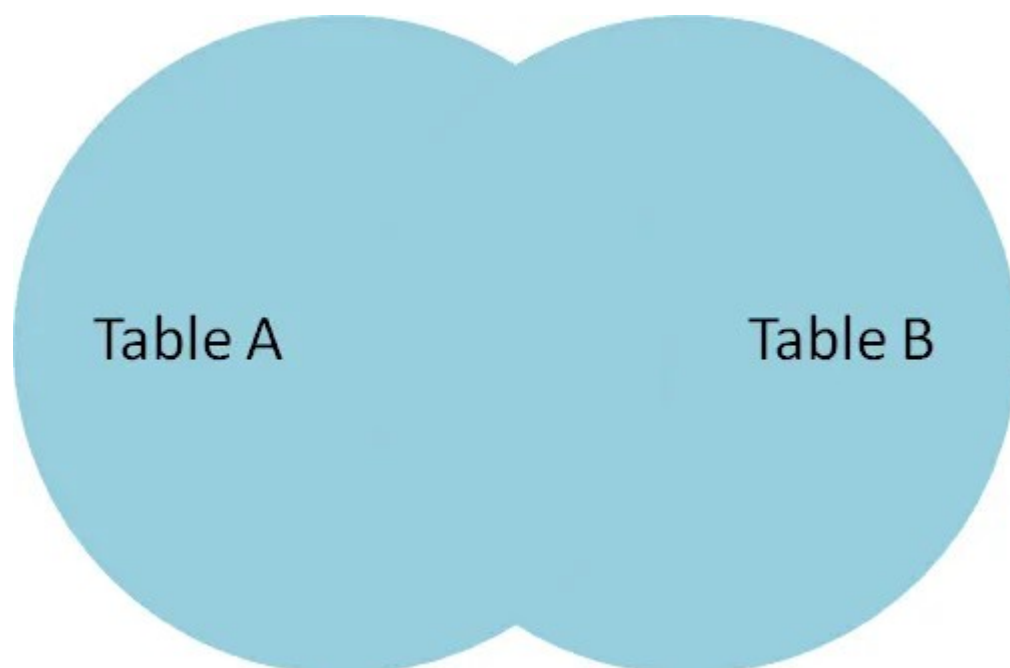
```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
	6

4. SQL FULL JOIN

FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.



Syntax

```
SELECT
table1.column1, table1.column2, table2.column1, ....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example: This example uses a FULL JOIN (simulated in SQLite) to return all rows from both tables. Matching records appear together, while non-matching records still show up with NULL values for the missing fields.

Query:

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output :

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	
	6

5. SQL Natural Join

A [Natural Join](#) is a type of INNER JOIN that automatically joins two tables based on columns with the same name and data type. It returns only the rows where the values in the common columns match.

- It joins tables using common columns with the same name.
- It returns only rows where values in those columns match.
- The common column appears only once in the result.

Example: Look at the two tables below:

Employee Table:

Emp_id	Emp_name	Dept_id
1	Ram	10
2	Jon	30
3	Bob	50

Department Table:

Dept_id	Dept_name
10	IT
30	HR
40	TIS

Example: Find all Employees and their respective departments.

```
SELECT
    Emp_name,
    Dept_name
FROM Employee
NATURAL JOIN Department;
```

Output:

Emp_name	Dept_name
Ram	IT
Jon	HR